SKALA

KDT Cloud

쿠버네티스 이해 및 앱 배포 (집중실습)

2026.08

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX



KDT Cloud - 쿠버네티스 이해 및 앱 배포

집중 실습

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX

집중실습 한눈에 보기

Day 1 실습

실습	본 과정	소요 무엇을 하나
실습 1	1,2,5장	40분 클러스터에 가기 전 – 로컬에서 먼저 실행해 본다
실습 2	1장	20분 조정 루프와 술래잡기 – 지워도 지워도 세 개이다
실습 3	2장	40분 Harbor 에 이미지 올리기 ← 여기가 관문이다
실습 4	3장	15분 라벨 하나로 트래픽을 훔친다
실습 5	5장	20분 권한의 경계 확인하기

Day 2 실습

실습	본 과정	소요 무엇을 하나
실습 6	5장	15분 매니페스트를 30초에 – 뼈대를 추출해 고쳐 쓴다
실습 7	7장	30분 프로브를 직접 흔들어 본다
실습 8	7장	20분 로그를 제대로 보는 법
실습 9	8장	30분 실패하는 배포 만들기
실습 10	9장	30분 Endpoints 에서 빠지는 순간

실습 준비. 이름 규칙과 접속

네임스페이스를 반 전체가 공유한다. 이름을 안 붙이면 옆 사람 것을 지운다

```
# ① 자기 식별자와 반 — 모든 실습에서 쓴다
export ME=p000                    # 반드시 자기 고유번호
export NS=class-6~10              # 자기 반

# ② 클러스터 접속
aws eks update-kubeconfig --name skala-2026 \
  --region ap-northeast-2 --profile skala-2026
kubectl config set-context --current --namespace=$NS
```

실습 매니페스트에는 이름 자리에 __ME__ 라는 자리표시자가 들어 있다.
render.sh 가 그 자리를 각자의 ME 값으로 채워서 출력한다.
shop-api 처럼 그냥 사용하면 옆 사람 것과 충돌한다. 더 위험한 것은 지울 때이다.
확인 없이 delete 를 치면 남의 것이 지워지고 되살릴 수 없다.
본문의 색이 셋으로 갈린다. 검정은 직접 치는 명령, 회색은 화면에 나오는 출력,
초록은 설명이다. 회색 줄은 따라 치지 않는다.

실습 준비. 이름 규칙과 접속

매니페스트는 눈으로 읽고 나서 적용한다

```
# ③ 이 클러스터는 풀 시크릿을 만들지 않는다
#    노드가 레지스트리에서 바로 받도록 구성되어 있다
kubectl get secret
No resources found in class-6 namespace.      <- 비어 있어도 정상이다

# ④ 매니페스트는 '읽고 나서' 적용한다
cd 샘플코드/k8s/skala-2026
./render.sh 107-pod.yaml                      <- 먼저 눈으로 본다
./render.sh 107-pod.yaml | kubectl apply -f -
```

다른 교재나 사내 다른 클러스터에서는 docker-registry 타입 시크릿을 만들어
imagePullSecrets 로 연결한다. 이 클러스터에는 그 단계가 없다.
render.sh 는 적용하지 않고 출력만 한다. 파이프로 apply 에 넘겨야 적용된다.

실습 1. 클러스터에 가기 전

로컬에서 먼저 실행해 본다. 여기서 안 되는 것은 클러스터에서도 안 된다

```
cd 샘플코드/shop-api
./gradlew test                                          <- 7 개 통과
docker build --platform linux/amd64 -t shop-api:1.0.0 .

docker run --rm -p 8080:8080 -p 8081:8081 \
  --memory=1g -e POD_NAME=local shop-api:1.0.0

curl -s localhost:8080/api/info | jq .
curl -s -o /dev/null -w '%{http_code}' localhost:8080/actuator/health
404                                                     <- 404 가 정상이다
```

--memory=1g 가 이 묶음의 핵심이다. 클러스터에서 적용할 limit 을 미리 흉내 낸다.
무제한으로만 실행하면 OOMKilled 를 클러스터에서 처음 만나게 되고,
그때는 원인 찾기가 훨씬 어렵다.
마지막 404 는 정상이다. 관리 포트를 8081 로 분리해 뒀기 때문이다.

실습 1. 클러스터에 가기 전

kubectl 을 손에 붙이고, 조정 루프를 눈으로 확인한다

```
-- kubectl 손에 붙이기 (~/.zshrc 에 넣는다) --
alias k=kubectl
alias kgp='kubectl get pod -o wide'
alias kn='kubectl config set-context --current --namespace'

-- 조정 루프 체험 — Pod 를 지워도 되살아난다 --
kubectl create deployment hello --image=nginx:1.27 --replicas=3
kubectl delete pod $(kubectl get pod -l app=hello -o name | head -1)
kubectl get pod                <- 여전히 3 개, 접미사가 바뀐 것이 하나
kubectl delete deployment hello
```

지워진 Pod 가 되살아난 것이 아니다. 완전히 새 Pod 가 만들어진 것이다.
이름 뒤의 랜덤 접미사가 바뀐 것을 보면 안다.
이것이 조정 루프이다. '세 개가 있어야 한다' 고 적어 두면 컨트롤러가 계속
그 상태로 맞춘다. 명령을 내린 것이 아니라 상태를 선언한 것이다.

실습 2. 조정 루프와 술래잡기

지워도 지워도 세 개이다. 터미널 두 개로 경주해 본다

```
-- 터미널 1 : 만들고 지켜본다 --
kubectl create deployment race-$ME --image=nginx:1.27 --replicas=3
kubectl get pod -l app=race-$ME -w
NAME                      READY   STATUS              AGE
race-p000-6f4b2-abcde   1/1     Terminating         32s
race-p000-6f4b2-pqrst   0/1     ContainerCreating   0s
race-p000-6f4b2-pqrst   1/1     Running             2s      <- 2 초 만에 복구
```

```
-- 터미널 2 : 계속 지운다 --
while true; do
  kubectl delete pod -l app=race-$ME --wait=false >/dev/null
  sleep 1
done
```

몇 번을 지워도 개수는 3 으로 돌아온다. 이길 수 없다. 지워진 Pod 가 되살아난 것이 아니라 매번 새 Pod 가 만들어진다. 접미사가 계속 바뀐다.
명령을 내린 것이 아니라 '세 개여야 한다' 고 적어 둔 것뿐이다.

실습 2. 조정 루프와 술래잡기

--cascade=orphan 으로 조정 루프의 사슬을 한 칸씩 끊는다

```
# ① Deployment 만 떼어 낸다 — ReplicaSet 과 Pod 는 살아남는다
kubectl delete deployment race-$ME --cascade=orphan
kubectl get rs,pod -l app=race-$ME          <- 그대로 3 개

# ② 이 상태에서 Pod 를 지우면? — 아직 ReplicaSet 이 돌본다
kubectl delete pod -l app=race-$ME
kubectl get pod -l app=race-$ME             <- 새 Pod 3 개. 여전히 진다

# ③ ReplicaSet 까지 떼어 낸다 — 이제 돌보는 컨트롤러가 없다
kubectl delete rs -l app=race-$ME --cascade=orphan
kubectl delete pod -l app=race-$ME
kubectl get pod -l app=race-$ME
No resources found                          <- 이번에는 안 되살아난다
```

조정 루프는 '누가 무엇을 돌보는가' 의 사슬이다. Deployment → ReplicaSet → Pod 로 소유가 이어져 있고, --cascade=orphan 은 그 사슬을 한 칸 끊는다.
사슬이 끊긴 Pod 는 중단되어도 아무도 다시 만들어 주지 않는다.
kubectl run 으로 만든 Pod 가 처음부터 이 상태이다. 노드가 멈추면 끝이다.

실습 3. Harbor 에 이미지 올리기

ECR 절차와 한 줄씩 대응된다. 로그인 대상에 https:// 를 붙이면 막힌다

```
# ① 로그인 — 호스트명만. https:// 를 붙이면 push 에서 401 이 난다
docker login skala-registry.skala-ai.com -u class-6

# ② 빌드 — 경로의 두 번째 칸이 Harbor 프로젝트 = 자기 반 이름
IMG=skala-registry.skala-ai.com/class-6/shop-$ME:1.0.0
docker build --platform linux/amd64 -t $IMG .    # 맥 M시리즈는 필수
docker push $IMG
```

docker 는 자격증명을 저장할 때 키를 문자열 그대로 사용한다. https:// 를 붙여
로그인하면 그 키로 저장되는데, push 는 이미지 이름에서 호스트명만 추출해
조회한다. 키가 안 맞으니 401 이다. 실습 중 401 의 1순위 원인이다.
--platform 을 생략하면 arm64 로 빌드되고 노드는 x86 이라 exec format error 가 발생한다.

실습 3. Harbor 에 이미지 올리기

클러스터는 풀 시크릿 없이 이미지를 받는다

```
# ③ 올린 이미지를 클러스터가 바로 받는지 확인한다
kubectl run probe-$ME --rm -it --restart=Never \
  --image=$IMG -- echo ok
ok                                                  <- 받아졌다

# 매니페스트에 imagePullSecrets 줄이 없다
grep -c imagePullSecrets 샘플코드/k8s/skala-2026/*.yaml
0
```

노드가 레지스트리에서 바로 받도록 구성되어 있어서 풀 시크릿이 필요 없다.
ECR 이라면 노드의 IAM 역할이 같은 일을 한다. 방식은 달라도 결과는 같다.
매니페스트에 적을 것이 없다.
사내 다른 클러스터로 옮기면 그때는 시크릿을 만들어 연결해야 한다.
그래서 ImagePullBackOff 가 발생하면 여기서는 경로·태그 오타를 먼저 본다.

실습 4. 라벨 하나로 트래픽을 훔친다

라벨을 떼면 ReplicaSet 이 '하나 모자라다' 며 새로 만든다

```
kubectl create deployment shop-$ME --image=nginx:1.27 --replicas=3
kubectl get pod -l app=shop-$ME --show-labels
NAME                       READY   STATUS    LABELS
shop-p000-7c9d4-abcde   1/1     Running   app=shop-p000,pod-template-hash=7c9d4
shop-p000-7c9d4-fghij   1/1     Running   app=shop-p000,pod-template-hash=7c9d4
shop-p000-7c9d4-klmno   1/1     Running   app=shop-p000,pod-template-hash=7c9d4

# 하나만 이름표를 뗀다
POD=$(kubectl get pod -l app=shop-$ME -o name | head -1)
kubectl label $POD app-

kubectl get pod --show-labels | grep -c shop-p000
4                                   <- 하나 줄었는데 전체는 늘었다
```

ReplicaSet 은 Pod 를 이름으로 세지 않는다. selector 와 맞는 라벨을 가진 Pod 가 몇 개인지만 센다. 이름표를 떼는 순간 그 Pod 는 ReplicaSet 의 눈에서 사라졌고, 그래서 '하나 모자라다' 며 새로 만들었다. 뗀 Pod 는 종료되지 않고 고아로 남는다. 문제 있는 Pod 를 서비스에서 제외하고 살려 둘 때 사용하는 기법이다.

실습 4. 라벨 하나로 트래픽을 훔친다

그 고아 Pod 에 다른 이름표를 붙이면 엉뚱한 Service 에 끼어든다

```
kubectl create deployment other-$ME --image=nginx:1.27
kubectl expose deployment other-$ME --port=80

kubectl get endpoints other-$ME
NAME          ENDPOINTS
other-p000    10.0.1.34:80                  <- 자기 Pod 하나

# 아까 고아가 된 Pod 에 other 의 이름표를 붙인다
kubectl label $POD app=other-$ME

kubectl get endpoints other-$ME
NAME          ENDPOINTS
other-p000    10.0.1.34:80,10.0.2.71:80     <- 남의 Pod 가 끼어들었다
```

Service 는 어떤 Deployment 에 속하는지 모른다. selector 와 맞는 라벨을 가진 Ready 인 Pod 를 전부 Endpoints 에 넣을 뿐이다. 문자열 일치가 전부이다.
그래서 라벨 오타 하나로 Endpoints 가 비고, 반대로 실수로 겹치면 엉뚱한 Pod 로 트래픽이 간다. 2일차 진단 카드 D3 이 바로 이 사고이다.

실습 5. 권한의 경계 확인하기

되는 것보다 '안 되는 것이 제대로 안 되는지' 를 먼저 확인한다

```
-- 되어야 하는 것 --
kubectl get nodes                    <- 클러스터 읽기 전용 권한이 있다
kubectl get sc                       <- ebs-sc(기본) · efs-sc

-- Forbidden 이 나와야 정상인 것 --
kubectl get pods -n class-7        <- 다른 반
kubectl create namespace my-test     <- 네임스페이스 생성

-- 에러를 만나기 전에 미리 물어본다 --
kubectl auth can-i create deployment                 <- yes
kubectl auth can-i create deployment -n class-7    <- no
kubectl auth can-i --list
```

Forbidden 메시지에 답이 다 있다. 누가(User) · 무엇을(resource) · 어디서(namespace).
RBAC 디버깅은 사실상 이 문장을 읽는 것이 전부이다.
can-i --list 를 지금 한 번 훑어 두면 나중에 Forbidden 을 만나도 당황하지 않는다.

실습 6. 매니페스트를 30초에

빈 파일에서 시작하지 않는다. 명령으로 뼈대를 추출해 고쳐 쓴다

```
# Deployment 뼈대
kubectl create deployment shop-$ME --image=nginx:1.27 \
  --dry-run=client -o yaml > deploy.yaml

# Service 뼈대 — 이미 있는 Deployment 에 맞춰서
kubectl expose deployment shop-$ME --port=80 --target-port=8080 \
  --dry-run=client -o yaml > svc.yaml

# ConfigMap
kubectl create configmap shop-config-$ME \
  --from-literal=APP_GREETING=hello --dry-run=client -o yaml > cm.yaml
```

--dry-run=client -o yaml 하나면 어떤 리소스든 올바른 뼈대가 나온다.
apiVersion 도 kind 도 selector 도 이미 맞게 들어가 있다.
expose 가 특히 편하다. Deployment 를 보고 selector 를 알아서 맞춰 주므로
라벨 오타로 Endpoints 가 비는 사고를 원천적으로 막는다.

실습 6. 매니페스트를 30초에

필드 이름은 클러스터에 물어보고, 바뀔 것을 보고 나서 적용한다

```
# 필드 이름이 기억 안 나면 클러스터에 물어본다
kubectl explain deployment.spec.template.spec.containers
kubectl explain deployment.spec.template.spec.containers.livenessProbe

# 고친 뒤 무엇이 바뀌는지 보고 적용
kubectl diff -f deploy.yaml
kubectl apply -f deploy.yaml
```

explain 은 인터넷 검색보다 정확하다. 지금 접속한 클러스터 버전을 기준으로
답하기 때문에, 버전이 안 맞아 없는 필드를 사용하는 실수가 없다.
diff 는 지금 클러스터 상태와 내 파일을 비교해 무엇이 바뀔지 보여 준다.
공유 네임스페이스에서는 특히 중요하다. 내가 모르는 사이에 누가 바꿔 놨을 수 있다.
손으로 타이핑하면 오타와 들여쓰기로 시간을 버린다.

실습 7. 프로브를 직접 흔들어 본다

readiness 를 내려도 재시작은 안 일어난다. 이것을 눈으로 확인한다

```
cd 샘플코드/k8s/skala-2026
export ME=p000
./render.sh 107-pod.yaml | kubectl apply -f -

# Downward API 로 주입된 값이 실제로 들어갔는지
kubectl exec shop-$ME -- env | grep -E 'POD_NAME|NODE_NAME'

# readiness 만 내린다
kubectl exec shop-$ME -- \
  curl -sX POST 'localhost:8080/api/unready?ready=false'
kubectl get pod shop-$ME
NAME        READY   STATUS    RESTARTS
shop-p000  0/1     Running   0            <- 0 이어야 한다
```

이 세 값의 조합이 정답이다. 트래픽은 끊겼지만 컨테이너는 중단되지 않았다.
의존성이 돌아오면 그대로 다시 받는다. RESTARTS 0 이 합격 조건이다.
둘을 같은 엔드포인트로 묶으면 여기서 재시작이 일어난다. DB 가 끊기면
전 복제본이 동시에 중단되는 '재시작 폭풍' 의 시작점이다.

실습 8. 로그를 제대로 보는 법

logs 는 옵션이 절반이다. 옵션을 알면 찾는 시간이 크게 줄어든다

```
kubectl logs deploy/shop-$ME                # Deployment 로 지정해도 된다
kubectl logs -f deploy/shop-$ME             # 실시간
kubectl logs --tail=50 deploy/shop-$ME      # 마지막 50줄만
kubectl logs --since=10m deploy/shop-$ME    # 최근 10분
kubectl logs --timestamps deploy/shop-$ME # 시각 붙여서

# 복제본이 여러 개일 때 — 라벨로 전부 한 번에
kubectl logs -f -l app=shop-$ME --all-containers --max-log-requests=6
```

Pod 이름 대신 deploy/이름 으로도 된다. 랜덤 접미사를 복사할 필요가 없다.
--tail 과 --since 를 실전에서 가장 자주 사용한다. 장애가 방금 발생했으면
--since=10m 으로 최근 것만 본다.
--max-log-requests 를 붙인 이유가 있다. Pod 가 많으면 기본 한도인 5 개를 넘어
오류가 발생한다. 복제본 수에 맞춰 올려 주면 된다.

실습 8. 로그를 제대로 보는 법

--previous 를 모르면 CrashLoopBackOff 를 영원히 못 푼다

```
# 죽은 컨테이너의 마지막 말 — CrashLoop 진단의 핵심
kubectl logs shop-$ME-xxxxx --previous

# 사이드카가 있으면 컨테이너를 고른다
kubectl logs shop-$ME-xxxxx -c app

# 에러만 추려 보기
kubectl logs --since=1h deploy/shop-$ME \
  | grep -iE 'error|exception' | tail -20
```

CrashLoopBackOff 는 떴다 중단되기를 반복하는 상태이다. 지금 보이는 컨테이너는 방금 만들어진 것이라 로그가 거의 없다. 단서는 직전에 종료된 컨테이너에 있고, --previous 가 그것을 보여 준다.
로그가 아예 비어 있으면 앱 문제가 아니다. Pending · ImagePullBackOff 는 컨테이너가 아직 없어서 로그가 없는 것이다. 그때는 describe 로 간다.

실습 9. 실패하는 배포 만들기

터미널 두 개로 한다. 하나는 계속 호출하고 하나는 배포한다

```
-- 터미널 1 : 끊김을 실시간으로 본다 --
kubectl port-forward svc/shop-$ME 8080:80 &
while true; do
  curl -s -o /dev/null -w "%{http_code} " localhost:8080/api/info
  sleep 0.3
done
200 200 200 200 200 200 200 200 200 200 200 200 200 200 200

-- 터미널 2 : 있지도 않은 태그로 배포 --
kubectl set image deploy/shop-$ME app=skala-registry.skala-ai.com/...:9.9.9
kubectl rollout status deploy/shop-$ME --timeout=60s    <- 멈춘다
```

성공하는 배포보다 실패하는 배포가 훨씬 많이 가르쳐 준다.
0.3 초마다 호출하면서 응답 코드를 찍어 두고, 다른 터미널에서 없는 태그로
배포한다. rollout status 는 타임아웃까지 기다리다 실패로 끝난다.

실습 9. 실패하는 배포 만들기

터미널 1 에 200 만 찍혀야 한다. 이것이 maxUnavailable: 0 이다

```
kubectl get pods
NAME                          READY   STATUS             RESTARTS
shop-p000-7c9d4f5b8-abcde   1/1     Running            0        <- 기존
shop-p000-6f4b2a1c9-xyzab   0/1     ImagePullBackOff   0        <- 새 것
```

```
kubectl rollout undo deploy/shop-$ME
deployment.apps/shop-p000 rolled back
```

새 Pod 는 ImagePullBackOff 인데 기존 Pod 는 그대로 Running 이다.
터미널 1 에는 200 만 찍힌다. 사용자는 아무것도 못 느꼈다.
기존 Pod 를 내리기 전에 새 Pod 가 Ready 가 되기를 기다리기 때문이다.
새 Pod 가 영원히 Ready 가 안 되니 기존 것을 영원히 안 내린다. 배포는 실패했지만
서비스는 멀쩡하다. 기본값 25% 였다면 용량이 75% 로 줄어든 채 멈춘다.

실습 10. Endpoints 에서 빠진다

Pod 는 Running 인데 503 이 발생하는 상황을 Endpoints 로 확인한다

```
kubectl scale deploy/shop-$ME --replicas=3
kubectl get endpoints shop-$ME -o jsonpath='{.subsets[0].addresses[*].ip}'
10.0.1.34 10.0.1.87 10.0.2.15                          <- IP 3 개

POD=$(kubectl get pod -l app=shop-$ME -o name | head -1)
kubectl exec $POD -- curl -sX POST 'localhost:8080/api/unready?ready=false'

kubectl get endpoints shop-$ME -o jsonpath='{.subsets[0].addresses[*].ip}'
10.0.1.87 10.0.2.15                                    <- IP 2 개

kubectl get pod $POD
NAME                READY   STATUS    RESTARTS
shop-p000-abcde     0/1     Running   0              <- 여전히 Running
```

진단 순서가 굳어진다. get pods 의 READY 열, get endpoints, describe 순서이다.
Running + READY 0/1 조합이 'Pod 는 떠 있는데 503' 의 정체이다.
Endpoints 가 비는 원인은 크게 둘이다. 지금 본 readiness 실패, 그리고
Service 의 selector 오타. describe svc 와 get pods --show-labels 로 가른다.



수고하셨습니다.

본 문서는 SK AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX